Problema

TecniHogar necesita un sistema de consola que permita administrar las órdenes de servicio técnico que se realizan a domicilio. Actualmente, la empresa maneja diferentes categorías de servicios, cada una con sus propias reglas para calcular el costo, determinar el tiempo estimado de atención, establecer la cantidad máxima de visitas y verificar si una cuadrilla puede atenderla.

El sistema deberá utilizar herencia para representar las características comunes de todas las órdenes y polimorfismo para ejecutar las reglas particulares de cada categoría. De esta manera, no será necesario preguntar constantemente qué tipo de orden se está procesando mediante estructuras como if, switch o instanceof. Cada objeto será responsable de aplicar sus propias reglas.

Además de administrar las órdenes, el programa deberá registrar cuadrillas, crear jornadas de trabajo, asignar órdenes, controlar las visitas realizadas, registrar los ingresos y producir reportes administrativos.

Requisitos funcionales

Información que el sistema debe conservar

El sistema debe mantener en memoria la información de todas las órdenes registradas. Cada orden debe conservar su código, cliente, dirección, horas estimadas, zona, estado actual, cantidad de visitas realizadas y categoría de servicio. Dependiendo de la categoría, también deberá conservar información adicional, como la cantidad de puntos de una instalación, el valor de los materiales de una instalación eléctrica certificada o la capacidad en BTU de un equipo de aire acondicionado.

También se debe conservar la información de las cuadrillas. Cada cuadrilla tiene un código único, una capacidad máxima de horas por jornada y una indicación que determina si cuenta con certificación en refrigeración.

El sistema debe conservar las jornadas creadas, la cuadrilla asignada a cada jornada, las órdenes incluidas, las horas acumuladas y el estado de la jornada.

Finalmente, debe mantener un registro separado de los ingresos producidos por el registro de órdenes y de los ingresos generados por visitas adicionales. Las órdenes iniciales solicitadas en el enunciado deben cargarse sin registrar ingresos.

Operaciones sobre las órdenes

El usuario podrá solicitar una cotización proporcionando la categoría y los datos correspondientes al servicio. Esta operación debe calcular y mostrar el costo y el

tiempo estimado de atención, pero no debe almacenar una orden ni modificar los ingresos.

Si el cliente acepta la cotización, el usuario podrá registrar la orden. En ese momento se deberá comprobar que el código no esté repetido, agregar la orden al sistema, colocarla en estado pendiente y registrar el ingreso correspondiente al costo completo del servicio.

El usuario también podrá consultar las órdenes registradas y observar sus datos, categoría, estado, visitas realizadas, costo y tiempo estimado de atención.

Operaciones sobre las cuadrillas

El usuario podrá registrar cuadrillas nuevas, siempre que su código sea válido y no se encuentre repetido. También podrá consultar todas las cuadrillas registradas, su capacidad máxima, su certificación y si tienen una jornada activa.

Operaciones sobre las jornadas

El usuario podrá crear una jornada seleccionando una cuadrilla disponible. La jornada se crea inicialmente en estado de preparación y sin órdenes asignadas.

Mientras la jornada permanezca en preparación, podrán agregarse órdenes pendientes que cumplan las restricciones de capacidad y compatibilidad. También será posible retirar órdenes, las cuales volverán al estado pendiente.

La jornada podrá comenzar únicamente cuando tenga por lo menos una orden. Al iniciar, todas sus órdenes pasarán al estado en atención. En ese momento se incrementará la cantidad de visitas de cada orden y se cobrará Q40.00 por aquellas cuya visita sea posterior a la primera.

Después de realizar la jornada, se deberá registrar el resultado de cada orden. El resultado puede ser una visita completada o una visita fallida. Cuando todas las órdenes tengan un resultado registrado, la jornada se cerrará automáticamente y la cuadrilla quedará disponible.

Operaciones de reportes

El sistema deberá generar un reporte con la cantidad de órdenes existentes por categoría y por estado. También debe mostrar el ingreso total, diferenciando los ingresos obtenidos por el registro de órdenes de los ingresos generados por visitas adicionales.

Asimismo, deberá calcular el ingreso acumulado de cada categoría de servicio. Para ello es conveniente conservar movimientos de ingreso que indiquen su monto, origen, orden y categoría.

El reporte de jornadas activas deberá mostrar todas las jornadas que todavía no estén cerradas, la cuadrilla utilizada, las horas asignadas y la capacidad disponible.

Condiciones de validez

Los códigos de las órdenes, cuadrillas y jornadas no pueden estar vacíos. Los códigos de las órdenes y cuadrillas deben ser únicos. Si también se utiliza un código para identificar una jornada, este deberá ser único dentro del sistema.

El nombre del cliente y la dirección deben contener información válida. Las horas estimadas deben ser mayores que cero y la zona debe corresponder a uno de los valores permitidos: metropolitana o periférica.

Una orden de diagnóstico no puede tener más de 1.5 horas estimadas. Una instalación debe tener una cantidad de puntos mayor que cero. Una instalación eléctrica certificada debe tener una cantidad positiva de puntos y un valor de materiales válido, mayor o igual que cero. Un mantenimiento de aire acondicionado debe indicar una capacidad entre 9,000 y 60,000 BTU y solamente puede registrarse en la zona metropolitana.

La capacidad máxima de una cuadrilla debe ser mayor que cero. Una orden solamente puede asignarse cuando está pendiente, no pertenece a otra jornada y su incorporación no provoca que se supere la capacidad de horas de la cuadrilla.

Para comprobar la capacidad se utilizan las horas estimadas originales de las órdenes. No deben utilizarse las horas facturables calculadas para cobrar una instalación.

Los mantenimientos de aire acondicionado solamente pueden asignarse a cuadrillas certificadas en refrigeración. Las demás categorías pueden ser atendidas por cualquier cuadrilla.

Una cuadrilla no puede tener más de una jornada sin cerrar. Una jornada cerrada tampoco puede recibir nuevas órdenes ni volver a iniciarse.

La jornada solo puede modificarse mientras se encuentra en preparación. Únicamente en ese estado se permite asignar y retirar órdenes. No puede iniciarse una jornada vacía y tampoco puede iniciarse más de una vez.

Una orden completada o cancelada no puede asignarse nuevamente. Una orden con visita fallida podrá regresar al estado pendiente solamente si todavía tiene visitas disponibles. Si alcanzó el máximo permitido para su categoría, deberá pasar al estado cancelada.

Toda operación inválida debe rechazarse mediante un mensaje claro y sin modificar la información previamente almacenada. Por ejemplo, si una orden excede la capacidad de la cuadrilla, no debe agregarse temporalmente para después eliminarla; primero deben comprobarse todas las condiciones y únicamente después realizar la asignación.

Diagnóstico

El diagnóstico tiene un costo fijo y no depende de la cantidad de horas. En la zona metropolitana cuesta Q150.00 y en la zona periférica cuesta Q225.00. Sus horas estimadas no pueden superar 1.5.

Instalación

El costo de una instalación está compuesto por una tarifa base según la zona y Q90.00 por cada hora facturable o fracción.

La tarifa base es de Q100.00 en la zona metropolitana y Q175.00 en la periférica. Para determinar las horas facturables se calculan primero las horas por puntos, multiplicando la cantidad de puntos por 0.75. Después se selecciona el mayor valor entre las horas estimadas y las horas por puntos. Finalmente, el resultado se redondea hacia arriba.

Instalación eléctrica certificada

La instalación eléctrica certificada utiliza inicialmente el mismo cálculo de una instalación normal. Al resultado se agregan Q120.00 por la emisión del certificado y una garantía equivalente al 5 % del valor declarado de los materiales.

Mantenimiento de aire acondicionado

El costo del mantenimiento se compone de la tarifa base según la zona, Q110.00 por cada hora estimada y un cargo fijo de Q80.00 por gas refrigerante.

Como este servicio solamente está permitido en la zona metropolitana, en la práctica utiliza la tarifa base metropolitana de Q100.00.

Visitas adicionales

La primera visita no produce un cobro adicional. A partir de la segunda visita, cada visita genera un cargo de Q40.00. Este ingreso debe registrarse cuando la jornada inicia, porque en ese momento la visita comienza realmente. Asignar una orden a una jornada no debe producir ingresos, ya que todavía puede ser retirada.

Tiempo estimado de atención

Una orden de diagnóstico tiene un tiempo estimado de un día hábil cuando pertenece a la zona metropolitana y de dos días hábiles cuando pertenece a la zona periférica.

Una instalación normal tiene un tiempo estimado de tres días hábiles en la zona metropolitana y cinco días hábiles en la zona periférica.

Una instalación eléctrica certificada tarda dos días más que una instalación normal de la misma zona. Por consiguiente, tarda cinco días en la zona metropolitana y siete días en la periférica.

El mantenimiento de aire acondicionado siempre tiene un tiempo estimado de dos días hábiles. No necesita diferenciar por zona porque solamente puede registrarse en la zona metropolitana.

Decisiones que dependen de la categoría

El cálculo del costo debe ser responsabilidad de cada clase concreta de orden. La clase abstracta OrdenServicio puede declarar el método abstracto calcularCosto(), pero las clases Diagnostico, Instalacion, InstalacionElectricaCertificada y MantenimientoAireAcondicionado deben implementar o sobrescribir este método. Esto permite que el sistema calcule el costo mediante polimorfismo.

El tiempo estimado de atención también depende de la categoría. Por esta razón, OrdenServicio debe declarar calcularDiasAtencion() como método abstracto y cada subclase debe proporcionar el resultado correspondiente.

La cantidad máxima de visitas debe ser decidida por cada categoría mediante un método polimórfico como obtenerMaximoVisitas(). El diagnóstico y la instalación normal devolverán tres; la instalación certificada devolverá dos; y el mantenimiento devolverá uno.

La compatibilidad con una cuadrilla debe determinarse mediante un método como esCompatibleCon(Cuadrilla cuadrilla). La implementación general puede devolver true, porque la mayoría de las órdenes puede ser atendida por cualquier cuadrilla. La

clase MantenimientoAireAcondicionado debe sobrescribirlo para comprobar que la cuadrilla esté certificada en refrigeración.

Las validaciones específicas de los datos también corresponden a cada categoría. El diagnóstico debe validar su límite de horas; la instalación debe validar los puntos; la instalación certificada debe validar el valor de los materiales; y el mantenimiento debe validar los BTU y la zona metropolitana.

Esta distribución permite que Jornada, ReporteServicio y SistemaTecniHogar trabajen con referencias de tipo OrdenServicio, sin preguntar cuál es la clase concreta de la orden.

Estados de una orden

Una orden recién registrada inicia en estado PENDIENTE. En este estado se encuentra disponible para ser asignada a una jornada.

Cuando la orden se agrega a una jornada en preparación, cambia al estado ASIGNADA. Si se retira antes de iniciar la jornada, regresa a PENDIENTE.

Cuando la jornada inicia, la orden cambia de ASIGNADA a EN_ATENCION. En ese momento se incrementa su contador de visitas. Si la nueva visita es posterior a la primera, se registra el cargo adicional de Q40.00.

Si el resultado de la visita es satisfactorio, la orden cambia de EN_ATENCION a COMPLETADA. Este es un estado final y la orden no puede volver a asignarse.

Si la visita falla y todavía quedan visitas disponibles, la orden cambia de EN_ATENCION a PENDIENTE. Esto permite asignarla a una nueva jornada.

Si la visita falla y con ella se alcanza el límite de la categoría, la orden cambia de EN_ATENCION a CANCELADA. Este también es un estado final.

El recorrido normal de una orden puede expresarse así:

PENDIENTE → ASIGNADA → EN_ATENCION → COMPLETADA

Cuando se retira una asignación:

ASIGNADA → PENDIENTE

Cuando una visita falla y todavía quedan intentos:

EN_ATENCION → PENDIENTE

Cuando se agota la última visita:

EN_ATENCION → CANCELADA

Estados de una jornada

Una jornada inicia en estado EN_PREPARACION. Durante este estado se pueden asignar y retirar órdenes.

Cuando el usuario solicita iniciar la jornada y esta contiene por lo menos una orden, cambia al estado INICIADA. Sus órdenes pasan al estado EN_ATENCION y se registran los cargos correspondientes a las visitas adicionales.

Mientras está iniciada se registran los resultados de cada orden. Cuando todas las órdenes tienen un resultado, la jornada cambia automáticamente al estado CERRADA.

Al cerrar la jornada, la cuadrilla queda libre y puede utilizarse para crear una nueva jornada. Una jornada cerrada no puede abrirse nuevamente ni aceptar modificaciones.

El ciclo de la jornada es:

Clases propuestas

Clase abstracta OrdenServicio

Esta es la superclase de todas las categorías. Su responsabilidad es almacenar los datos comunes, controlar el estado y las visitas, y definir mediante métodos abstractos las decisiones que dependen de cada servicio.

Atributos:

- private String codigo: identificador único de la orden.
- private String nombreCliente: nombre de la persona que solicita el servicio.
- private String direccion: lugar donde se realizará el servicio.
- private double horasEstimadas: duración estimada del trabajo.
- private Zona zona: zona metropolitana o periférica.
- private EstadoOrden estado: situación actual de la orden.
- private int visitasRealizadas: cantidad de visitas iniciadas.

- private Jornada jornadaAsignada: jornada a la que pertenece actualmente o null si no está asignada.

El constructor de OrdenServicio recibe el código, nombre del cliente, dirección, horas estimadas y zona. Debe validar los datos comunes y colocar inicialmente la orden en estado pendiente, con cero visitas y sin jornada asignada.

El método calcularCosto() será público y abstracto, y devolverá un double. El método calcularDiasAtencion() será público y abstracto, y devolverá un int. El método obtenerMaximoVisitas() devolverá el límite correspondiente a la categoría. El método esCompatibleCon(Cuadrilla cuadrilla) permitirá comprobar si una cuadrilla puede atender la orden.

También deberá contener métodos protegidos o con visibilidad controlada para marcar la orden como asignada, retirarla, iniciar una visita y registrar su resultado. Estos métodos deben proteger las transiciones de estado.

Clase Diagnostico

Esta clase hereda de OrdenServicio y representa la revisión inicial de un problema.

Atributos:

- No necesita atributos adicionales.

Su constructor recibe los datos comunes y los envía al constructor de OrdenServicio mediante super(...). Antes de terminar la construcción debe garantizar que las horas no superen 1.5.

Sobrescribe calcularCosto() para devolver Q150.00 o Q225.00 según la zona. Sobrescribe calcularDiasAtencion() para devolver uno o dos días y obtenerMaximoVisitas() para devolver tres.

Clase Instalacion

Esta clase hereda de OrdenServicio y representa una instalación normal.

Atributos:

- private int cantidadPuntos: número de puntos que deben instalarse.

Su constructor recibe los datos comunes y la cantidad de puntos. Los datos comunes se envían al constructor de la superclase mediante super(...), mientras que la cantidad de puntos se valida y almacena en esta clase.

El método calcularHorasPorPuntos() multiplica la cantidad de puntos por 0.75. El método calcularHorasFacturables() obtiene el mayor valor entre las horas estimadas y las horas por puntos, y lo redondea hacia arriba.

El método calcularCosto() utiliza la tarifa base de la zona y agrega Q90.00 por cada hora facturable. calcularDiasAtencion() devuelve tres o cinco días según la zona y obtenerMaximoVisitas() devuelve tres.

Clase InstalacionElectricaCertificada

Esta clase hereda de Instalacion porque una instalación certificada cumple todas las reglas y posee todos los datos de una instalación normal, pero agrega el certificado, la garantía y una política diferente de atención y visitas.

Atributos:

- private double valorMateriales: valor declarado de los materiales utilizados.

Su constructor recibe los datos comunes, la cantidad de puntos y el valor de los materiales. Mediante super(...) permite que Instalacion y OrdenServicio inicialicen sus respectivas partes del objeto.

El método calcularCosto() debe invocar super.calcularCosto() para reutilizar el cálculo de la instalación normal y después agregar Q120.00 y el 5 % del valor de los materiales.

El método calcularDiasAtencion() puede invocar super.calcularDiasAtencion() y sumar dos días. El método obtenerMaximoVisitas() se sobrescribe para devolver dos.

Clase MantenimientoAireAcondicionado

Esta clase hereda directamente de OrdenServicio.

Atributos:

- private int capacidadBTU: capacidad del equipo, entre 9,000 y 60,000 BTU.

Su constructor valida el rango de los BTU y comprueba que la zona sea metropolitana. El método calcularCosto() suma la tarifa base, Q110.00 por cada hora estimada y Q80.00 por el gas refrigerante.

El método calcularDiasAtencion() siempre devuelve dos. El método obtenerMaximoVisitas() devuelve uno. El método esCompatibleCon(Cuadrilla cuadrilla) se sobrescribe para retornar true únicamente cuando la cuadrilla posee certificación en refrigeración.

Clase Cuadrilla

Esta clase representa al equipo de técnicos que trabaja durante una jornada.

Atributos:

- private String codigo: identificador único de la cuadrilla.
- private double capacidadMaximaHoras: total de horas que puede trabajar en una jornada.
- private boolean certificadaRefrigeracion: indica si puede realizar mantenimientos de aire acondicionado.
- private Jornada jornadaActiva: jornada sin cerrar que utiliza la cuadrilla o null si está disponible.

El constructor recibe el código, la capacidad máxima y la certificación. Debe validar que el código no esté vacío y que la capacidad sea positiva.

El método estaDisponible() indica si la cuadrilla puede utilizarse. Los métodos asignarJornada() y liberarJornada() deben controlar la relación con una jornada activa.

Clase Jornada

Esta clase administra un grupo de órdenes atendidas por una misma cuadrilla.

Atributos:

- private String codigo: identificador de la jornada.
- private Cuadrilla cuadrilla: cuadrilla responsable.
- private ArrayList<OrdenServicio> ordenes: órdenes incluidas en la jornada.
- private EstadoJornada estado: estado actual de la jornada.
- private Set<String> resultadosRegistrados: códigos de órdenes que ya tienen resultado.

El constructor recibe el código y la cuadrilla. La lista de órdenes se crea vacía y el estado inicial es EN_PREPARACION.

El método calcularHorasAsignadas() suma las horas estimadas de las órdenes. calcularCapacidadDisponible() resta las horas asignadas a la capacidad máxima.

El método asignarOrden() comprueba el estado de la jornada, el estado de la orden, la capacidad disponible, la compatibilidad y que la orden no pertenezca a otra

jornada. El método retirarOrden() elimina una orden mientras la jornada esté en preparación.

El método iniciar() verifica que existan órdenes y cambia su estado. El registro de los cargos de visitas adicionales debe coordinarse con SistemaTecniHogar para que los movimientos económicos se creen únicamente después de iniciar correctamente la jornada.

El método registrarResultado() comunica a la orden si la visita fue completada o fallida. Cuando todas las órdenes tienen resultado, la jornada se cierra y libera la cuadrilla.

Clase MovimientoIngreso

Esta clase representa un ingreso real de la empresa. Permite separar los ingresos por registro de los producidos por visitas adicionales y evita confundir una cotización con un cobro.

Atributos:

- private String codigoOrden: orden que produjo el ingreso.
- private CategoriaServicio categoria: categoría de la orden.
- private TipoIngreso tipo: registro de orden o visita adicional.
- private double monto: cantidad ingresada.

La clase debe ser inmutable. Sus atributos se asignan en el constructor y posteriormente solo se consultan mediante métodos de acceso.

Clase Cotizacion

Esta clase representa el resultado temporal de cotizar un servicio. No debe agregarse a la colección de órdenes ni producir movimientos de ingreso.

Atributos:

- private CategoriaServicio categoria: categoría solicitada.
- private double costo: costo calculado.
- private int diasAtencion: tiempo estimado.
- private String descripcion: resumen de los datos cotizados.

La cotización puede construirse a partir de una orden temporal válida. Después de calcular el costo y los días, el objeto temporal se descarta sin modificar el sistema.

Clase SistemaTecniHogar

Esta clase coordina las operaciones principales y conserva las colecciones del programa.

Atributos:

- private ArrayList<OrdenServicio> ordenes: órdenes registradas.
- private ArrayList<Cuadrilla> cuadrillas: cuadrillas registradas.
- private ArrayList<Jornada> jornadas: jornadas creadas.
- private ArrayList<MovimientoIngreso> movimientos: ingresos confirmados.

Esta clase debe buscar objetos por código, verificar códigos repetidos, registrar órdenes, registrar cuadrillas, crear jornadas y generar reportes. También debe garantizar que los movimientos de ingreso se creen solamente cuando se registra una orden aceptada o cuando comienza una visita adicional.

Las cotizaciones, asignaciones retiradas y operaciones rechazadas no deben llamar al método que registra movimientos económicos.

Clase ReporteServicio

Esta clase se encarga de procesar la información almacenada y presentar los reportes.

Atributos:

- private SistemaTecniHogar sistema: sistema del cual obtiene los datos.

Sus métodos deben contar órdenes por categoría y estado, sumar ingresos por tipo, calcular ingresos por categoría y mostrar las jornadas que todavía no se encuentran cerradas.

Los reportes deben apoyarse en los métodos polimórficos de las órdenes o en su categoría, sin utilizar comprobaciones con instanceof.

Clase Main

Esta clase contiene el punto de inicio y administra el menú de consola.

Atributos:

- private static Scanner scanner: permite leer los datos ingresados.
- private static SistemaTecniHogar sistema: objeto principal de la aplicación.

El método main(String[] args) debe crear el sistema, cargar los datos iniciales y mostrar el menú repetidamente hasta que el usuario elija salir.

También puede contener métodos privados para solicitar datos, leer números de manera segura y atender cada opción del menú. Las entradas con formato incorrecto deben manejarse sin finalizar el programa.

Enumeraciones necesarias

Zona debe contener los valores METROPOLITANA y PERIFERICA.

EstadoOrden debe contener PENDIENTE, ASIGNADA, EN_ATENCION, COMPLETADA y CANCELADA.

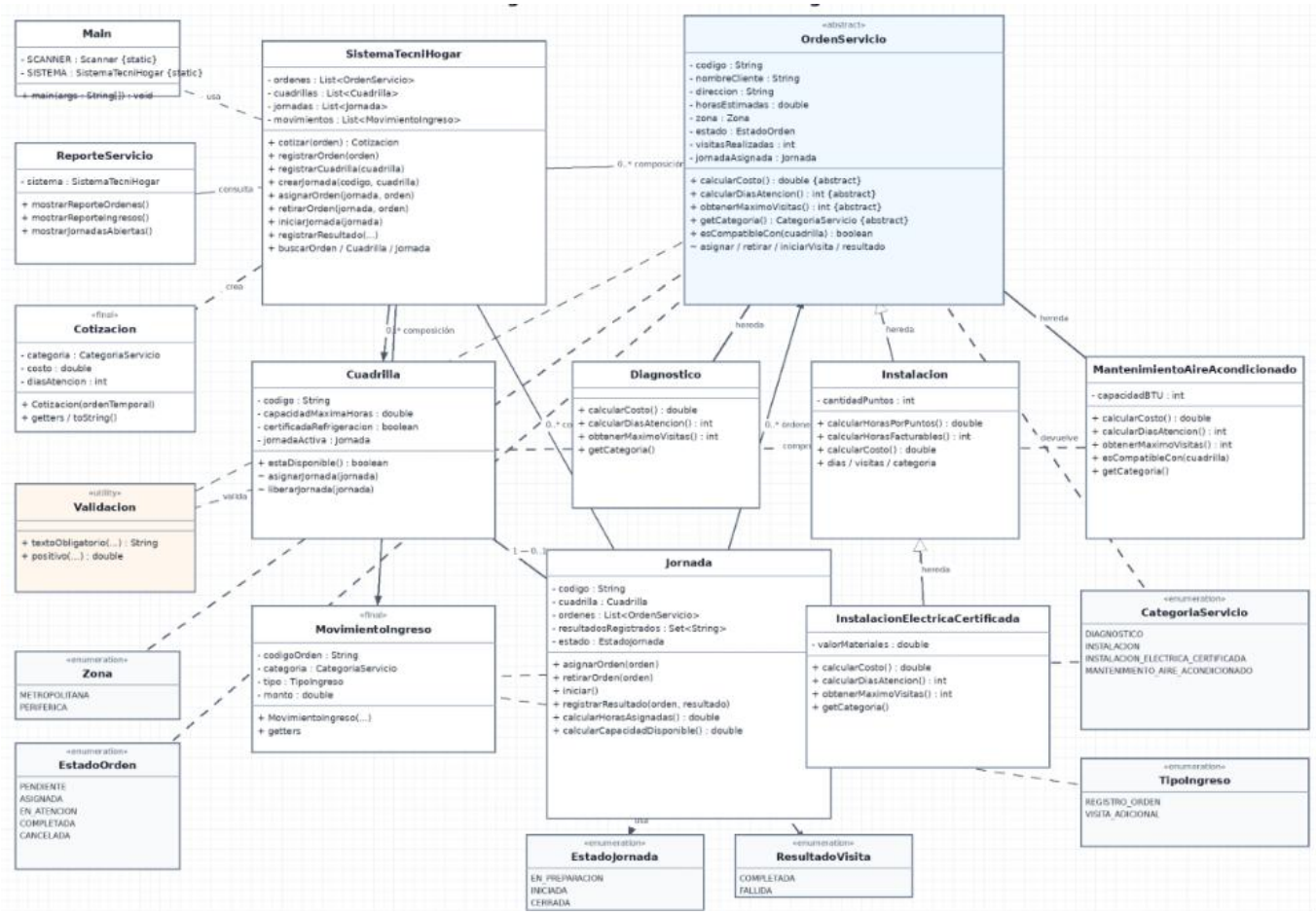
EstadoJornada debe contener EN_PREPARACION, INICIADA y CERRADA.

ResultadoVisita debe contener COMPLETADA y FALLIDA.

TipoIngreso debe contener REGISTRO_ORDEN y VISITA_ADICIONAL.

CategoriaServicio puede contener DIAGNOSTICO, INSTALACION, INSTALACION_ELECTRICA_CERTIFICADA y MANTENIMIENTO_AIRE_ACONDICIONADO.

Esta enumeración facilita la agrupación de los reportes, aunque las reglas de negocio deben permanecer en las clases y no dentro de un switch general.


Main
- SCANNER : Scanner {static}
- SISTEMA : SistemaTecniHogar {static}
+ main(args : String[]) : void
usa
SistemaTecniHogar
- ordenes : List<OrdenServicio>
- cuadrillas : List<Cuadrilla>
- jornadas : List<Jornada>
- movimientos : List<MovimientoIngreso>
+ cotizar(orden) : Cotizacion
+ registrarOrden(orden)
+ registrarCuadrilla(cuadrilla)
+ crearJornada(codigo, cuadrilla)
+ asignarOrden(jornada, orden)
+ retirarOrden(jornada, orden)
+ iniciarJornada(jornada)
+ registrarResultado(...)
+ buscarOrden / Cuadrilla / Jornada
«abstract»
OrdenServicio
- codigo : String
- nombreCliente : String
- direccion : String
- horasEstimadas : double
- zona : Zona
- estado : EstadoOrden
- visitasRealizadas : int
- jornadaAsignada : Jornada
+ calcularCosto() : double {abstract}
+ calcularDiasAtencion() : int {abstract}
+ obtenerMaximoVisitas() : int {abstract}
+ getCategoria() : CategoriaServicio {abstract}
+ esCompatibleCon(cuadrilla) : boolean
~ asignar / retirar / iniciarVisita / resultado
0..* composición
ReporteServicio
- sistema : SistemaTecniHogar
+ mostrarReporteOrdenes()
+ mostrarReporteIngresos()
+ mostrarJornadasAbiertas()
consulta
crea
«final»
Cotizacion
- categoria : CategoriaServicio
- costo : double
- diasAtencion : int
+ Cotizacion(ordenTemporal)
+ getters / toString()
0..* composición
hereda
hereda
hereda
Cuadrilla
- codigo : String
- capacidadMaximaHoras : double
- certificadaRefrigeracion : boolean
- jornadaActiva : Jornada
+ estaDisponible() : boolean
~ asignarJornada(jornada)
~ liberarJornada(jornada)
Diagnostico
+ calcularCosto() : double
+ calcularDiasAtencion() : int
+ obtenerMaximoVisitas() : int
+ getCategoria()
Instalacion
- cantidadPuntos : int
+ calcularHorasPorPuntos() : double
+ calcularHorasFacturables() : int
+ calcularCosto() : double
+ dias / visitas / categoria
MantenimientoAireAcondicionado
- capacidadBTU : int
+ calcularCosto() : double
+ calcularDiasAtencion() : int
+ obtenerMaximoVisitas() : int
+ esCompatibleCon(cuadrilla)
+ getCategoria()
devuelve
«utility»
Validacion
+ textoObligatorio(...) : String
+ positivo(...) : double
valida
1 — 0..1
Jornada
- codigo : String
- cuadrilla : Cuadrilla
- ordenes : List<OrdenServicio>
- resultadosRegistrados : Set<String>
- estado : EstadoJornada
+ asignarOrden(orden)
+ retirarOrden(orden)
+ iniciar()
+ registrarResultado(orden, resultado)
+ calcularHorasAsignadas() : double
+ calcularCapacidadDisponible() : double
hereda
«final»
MovimientoIngreso
- codigoOrden : String
- categoria : CategoriaServicio
- tipo : TipoIngreso
- monto : double
+ MovimientoIngreso(...)
+ getters
InstalacionElectricaCertificada
- valorMateriales : double
+ calcularCosto() : double
+ calcularDiasAtencion() : int
+ obtenerMaximoVisitas() : int
+ getCategoria()
«enumeration»
CategoriaServicio
DIAGNOSTICO
INSTALACION
INSTALACION_ELECTRICA_CERTIFICADA
MANTENIMIENTO_AIRE_ACONDICIONADO
«enumeration»
Zona
METROPOLITANA
PERIFERICA
«enumeration»
EstadoOrden
PENDIENTE
ASIGNADA
EN_ATENCION
COMPLETADA
CANCELADA
«enumeration»
TipoIngreso
REGISTRO_ORDEN
VISITA_ADICIONAL
tiene
«enumeration»
EstadoJornada
EN_PREPARACION
INICIADA
CERRADA
«enumeration»
ResultadoVisita
COMPLETADA
FALLIDA